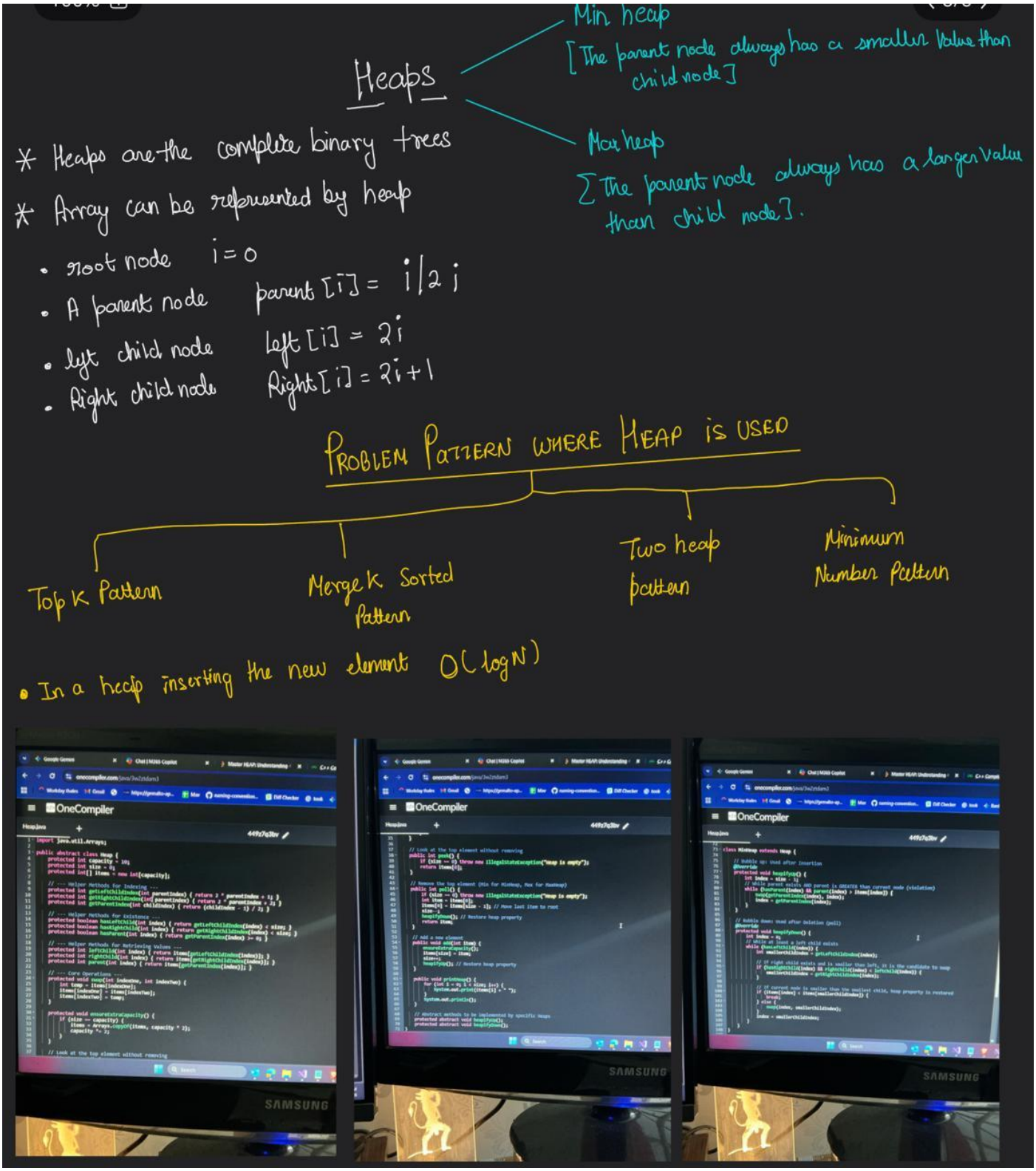
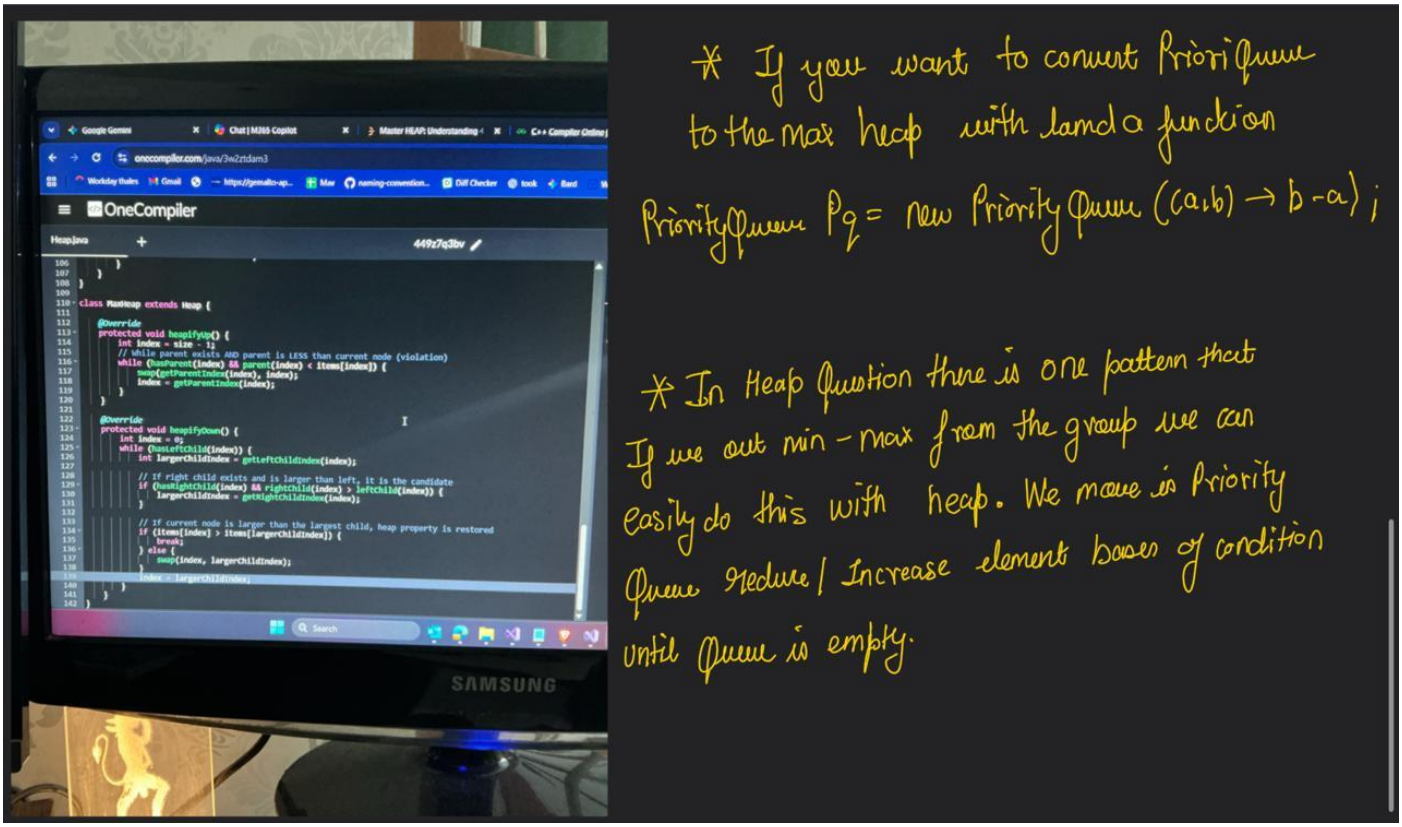
Heap Notes By Aman





The objective of this notes is to understand the basics of Heap, time complexities, and identify patterns when to use Heap as a data structure.

Heap questions are one of the most common questions frequently asked in interviews.

The confidence in HEAP data structure is guranteed if you finish below mentioned 23 questions.

What is Heap?

1. It is mainly used to represent a priority queue.
2. It is represented as a Binary Tree (a tree structure where a node of a tree has a maximum of two child nodes). Heaps are complete binary trees.
3. A simple array can be used to represent a Heap where array indices refer to the node position in the tree.
4. Parent and child nodes can be accessed with indices:
    - A root node | i = 0, the first item of the array
    - A parent node | parent(i) = i / 2
    - A left child node | left(i) = 2i
    - A right child node | right(i)=2i+1

5. Two type of Heaps — Min Heap, Max Heap
   Min Heap — the parent node always has a smaller value than the child nodes.
   Max Heap — the parent node is always larger than the child node value.
6. Usually, when a type is not mentioned, it refers to the MinHeap. Python Heap has minHeap as default.
7. In python, theheapqmodule provides the basic features for Heap data structure.
8. minHeap are used in tasks related to scheduling or assignment. A more detailed explanation is under the Patterns section below.

1. Array Representation (The "Memory" View)

In interviews, you might be asked: *"How is a heap actually stored in memory if it's not using nodes and pointers like a Tree?"*

Heaps are almost always implemented using an **Array**. Because a heap is a **Complete Binary Tree**, there are no "gaps," which makes array mapping very efficient.

For any element at index `i`:

- **Left Child:** $2i + 1$
- **Right Child:** $2i + 2$
- **Parent:** $(i - 1)/2$

Why this matters: Some interviewers will ask you to implement a Priority Queue from scratch without using Java's built-in library. Knowing these formulas is the only way to do it.

2. Heapify: $O(N)$ vs. $O(N \log N)$

This is a very common "trap" question in technical rounds.

- **Repeated Insertion:** If you take an empty heap and add N elements one by one, the complexity is $O(N \log N)$.
- **Heapify (Bottom-Up):** If you already have an array of N elements and you "heapify" it all at once (using `new PriorityQueue(List)` in Java), the complexity is actually $O(N)$.

The Interview Tip: If you are given a static array and asked to find the Top K elements, it is faster to "Heapify" the whole array first ($O(N)$) and then poll K times, rather than inserting one by one.

3. Time & Space Complexity Cheat Sheet

You must memorize these for your big-O analysis:

Operation	Complexity	Reason
Insert (add/offer)	$O(\log N)$	Must "bubble up" to maintain heap property.
Remove Min/Max (poll)	$O(\log N)$	Must "bubble down" the new root.
Peek	$O(1)$	Root is always at index 0.
Search	$O(N)$	It's not a BST; there is no order between left and right children.
Space	$O(N)$	To store the elements in the array.

Heap Patterns

1. Top K Elements Pattern

Used for finding the K largest or smallest elements in an unsorted collection.

- **Logic:** To find the **K Largest**, keep a **Min-Heap** of size K. If the next element is larger than the root, replace the root.
- **Time Complexity:** $O(N \log K)$

```java
import java.util.PriorityQueue;

public class TopK {
    public int findKthLargest(int[] nums, int k) {
        // Min-Heap (default in Java)
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.add(num);
            // If heap size exceeds k, remove the smallest element
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        // The root is the Kth largest element
        return minHeap.peek();
    }
}
```

2. Two Heaps Pattern

Used when you need to find a median or a split point in a dynamic stream of numbers.

- **Logic:**
    1. **Max-Heap (Left side):** Stores the smaller half of numbers.
    2. **Min-Heap (Right side):** Stores the larger half of numbers.
- **Balancing:** The size difference between heaps should never be more than 1.

Example: Find Median from Data Stream

Java

```java
import java.util.Collections;
import java.util.PriorityQueue;

class MedianFinder {
    private PriorityQueue<Integer> small = new PriorityQueue<>(Collections.revers
    private PriorityQueue<Integer> large = new PriorityQueue<>(); // Min-Heap

    public void addNum(int num) {
        small.add(num);
        // Balance: Move largest of small to large
        large.add(small.poll());

        // Maintain size property
        if (small.size() < large.size()) {
            small.add(large.poll());
        }
    }

    public double findMedian() {
        if (small.size() > large.size()) return small.peek();
        return (small.peek() + large.peek()) / 2.0;
    }
}
```

3. Merge K Sorted Pattern

Used to merge multiple sorted arrays, lists, or linked lists into a single sorted output.

- **Logic:** Push the first element of every list into a Min-Heap. Pop the minimum, and then push the next element from the *same* list that the popped element belonged to.

Example Snippet (Merging Arrays)

Java

```java
import java.util.PriorityQueue;

class Node {
    int val, listIdx, elementIdx;
    Node(int v, int li, int ei) { val = v; listIdx = li; elementIdx = ei; }
}

public class MergeKSorted {
    public int[] merge(int[][] lists) {
        PriorityQueue<Node> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val

        // Initialize heap with first element of each list
        for (int i = 0; i < lists.length; i++) {
            if (lists[i].length > 0)
                minHeap.add(new Node(lists[i][0], i, 0));
        }

        // Process heap and add next elements
        while (!minHeap.isEmpty()) {
            Node curr = minHeap.poll();
            // Process curr.val (e.g., add to result list)

            if (curr.elementIdx + 1 < lists[curr.listIdx].length) {
                minHeap.add(new Node(lists[curr.listIdx][curr.elementIdx + 1],
                                     curr.listIdx, curr.elementIdx + 1));
            }
        }
        return new int[0]; // Placeholder
    }
}
```

4. Frequency Sort / Reorganize Pattern

Used for problems involving character frequencies, like "Top K Frequent Elements" or "Rearrange string so no two identical characters are adjacent."

- **Logic:** Use a `HashMap` to count frequencies, then push the `Map.Entry` or a custom object into a **Max-Heap** based on the frequency count.

Example: Top K Frequent Elements

Java

```java
import java.util.*;

public class FrequencyPattern {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int n : nums) countMap.put(n, countMap.getOrDefault(n, 0) + 1);

        // Max-Heap based on frequency
        PriorityQueue<Map.Entry<Integer, Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());

        maxHeap.addAll(countMap.entrySet());

        int[] res = new int[k];
        for (int i = 0; i < k; i++) {
            res[i] = maxHeap.poll().getKey();
        }
        return res;
    }
}
```

Quick Cheat Sheet for Java Developers

Task	Code Snippet
Default Min-Heap	`new PriorityQueue<Integer>()`
Max-Heap	`new PriorityQueue<>(Collections.reverseOrder())`
Heap with Custom Object	`new PriorityQueue<>((a, b) -> a.val - b.val)`
Add Element	`.offer(element)` or `.add(element)`
Remove Top	`.poll()`
View Top	`.peek()`

5. When NOT to use a Heap

Knowing when a tool is wrong is just as important as knowing when it's right.

- **If you need to find the "closest" value to X:** Use a **TreeSet** or **TreeMap** (BST). Heaps only know about the absolute Min/Max.
- **If you need to print elements in sorted order frequently:** Use a **TreeSet**. Extracting everything from a heap to see it sorted destroys the heap (you'd have to rebuild it).
- **If you need to search for an element:** Use a **HashSet**. Searching in a heap is a slow $O(N)$ operation.

Problem Patterns where HEAP is used

9. Based on my understanding, different questions where HEAP is common data structure to use can be categorized in following 4 categories:
10. Top K Pattern
11. Merge K Sorted Pattern
12. Two Heaps Pattern
13. Minimum Number Pattern
14. All questions under one patterns has some similarities in terms of using HEAP as a data structure. Completing these questions would gurantee you mastery on the HEAP data structure. Below list includes some of the most common questions asked in most of the companies.

Top K Pattern

15. LC #215 - Kth largest number in an array
    LC #973 - K closest points to origin
    LC #347 - Top k frequent elements/numbers
    LC #692 - Top k frequent words
    LC #264 - Ugly Number II
    LC #451 - Frequency Sort
    LC #703 - Kth largest number in a stream
    LC #767 - Reorganize String
    LC #358 - Rearrange string K distance apart
    LC #1439 - Kth smallest sum of a matrix with sorted rows

16. Merge K sorted pattern

17. LC #23 - Merge K sorted
    LC #373 - K pairs with the smallest sum
    LC #378 - K smallest numbers in M-sorted lists

18. Two Heaps Pattern

19. LC #295 - Find median from a data stream
    LC #480 - Sliding window Median
    LC #502 - Maximize Capital/IPO

20. Minimum number Pattern

21. LC #1167 - Minimum Cost to connect sticks/ropes
    LC #253 - Meeting Rooms II
    LC #759 - Employee free time
    LC #857 - Minimumcost to hire K workers
    LC #621 - Minimum number of CPU (Task scheduler)
    LC #871 - Minimum number of Refueling stops